
PROYECTO 1 – CARD CLASH: IPC2 EDITION

202404856 – Angel Emanuel Rodriguez Corado

Resumen

En el presente proyecto se detalla el desarrollo de una simulación de un juego de cartas, con un enfoque primordial en la implementación de Tipos de Datos Abstractos (TDA). La concepción y construcción de la aplicación se realizó bajo los principios de la Programación Orientada a Objetos (POO), utilizando estructuras de datos personalizadas. Se emplean listas circulares simplemente enlazadas para la gestión del mazo, pilas (PilaCartas) para representar las manos de los jugadores y el historial de cartas en mesa, y una ListaGenerica doblemente enlazada para manejar jugadores y configuraciones.

Un aspecto fundamental del proyecto es la exclusión de listas nativas de Python para las operaciones principales del juego, demostrando el control y la eficiencia que los TDA ofrecen. La interfaz gráfica se implementó con la biblioteca tkinter, asegurando una interacción fluida con el usuario. Para la persistencia y carga de información, se utilizan archivos XML, tanto para las configuraciones de partida como para el registro de los pasos del juego. Finalmente, Graphviz se integra para generar representaciones gráficas del estado de la aplicación.

Palabras clave

Tipos de Datos Abstractos, Programación Orientada a Objetos, Python, archivos XML, Listas Enlazadas

Abstract

This project details the development of a simulation of a card game, with a primary focus on the implementation of Abstract Data Types (ADT). The conception and construction of the application was carried out under the principles of Object Oriented Programming (OOP), using custom data structures. Simply linked circular lists are used for deck management, stacks (PilaCartas) to represent players' hands and the history of cards on the table, and a doubly linked ListaGenerica to manage players and configurations.

A key aspect of the project is the exclusion of native Python lists for core game operations, demonstrating the control and efficiency that TDAs offer. The graphical interface was implemented with the tkinter library, ensuring a smooth interaction with the user. For persistence and information loading, XML files are used, both for game configurations and for logging game steps. Finally, Graphviz is integrated to generate graphical representations of the application state.

Keywords

Abstract Data Types, Object-Oriented Programming, Python, XML files, Linked Lists

Introducción

Este proyecto aborda la simulación de un juego de cartas, sirviendo como demostración práctica de principios clave de ingeniería de software. En un entorno digitalizado, modelar sistemas complejos eficientemente es vital. Se prioriza la aplicación de Tipos de Datos Abstractos (TDA) y Programación Orientada a Objetos (POO) para construir una solución robusta.

La implementación destaca por su compromiso con la abstracción, desarrollando estructuras como listas circulares para el mazo y pilas para las manos. Esto permite una comprensión profunda de los mecanismos internos y adapta el sistema. Se gestionan los datos de forma estructurada con XML y se visualiza el estado con Graphviz. El objetivo es evidenciar que el dominio de los TDA optimiza el rendimiento y fomenta software mantenible y comprensible.

Desarrollo del tema

El desarrollo y la ejecución de una simulación de un juego de cartas ofrecen un entorno perfecto para examinar el potencial de los Tipos de Datos Abstractos (TDA) y la Programación Orientada a Objetos (POO). Este trabajo no solo pretende replicar la lógica del juego, sino que también intenta crear una estructura arquitectónica sólida y ampliable, asegurando la eficiencia y la fácil mantenimiento del software.

a) Fundamentos de los Tipos de Datos Abstractos (TDA)

Los Tipos de Datos Abstractos son esenciales para un diseño de software efectivo. A diferencia de las estructuras de datos concretas, tales como listas o diccionarios que ya existen en un lenguaje de

programación, un TDA establece un conjunto de procedimientos y el funcionamiento de un tipo de dato, sin detallar cómo se implementan internamente. En el marco de este proyecto, optar por TDAs personalizados en lugar de usar las estructuras nativas de Python trae consigo varios beneficios clave:

Encapsulamiento y Abstracción: Los TDAs permiten ocultar los detalles de su implementación, proporcionando únicamente una interfaz bien definida. Esto simplifica el sistema y evita el acceso o manipulación indebida de los datos.

Flexibilidad y Reusabilidad: Al separar la interfaz de la implementación, se facilita la modificación de las estructuras de datos sin afectar al resto del sistema. Además, los TDAs pueden ser reutilizados en diferentes situaciones.

Control de Errores: Al establecer claramente las operaciones que se pueden realizar, los TDAs permiten incorporar métodos de validación y manejo de errores específicos para ese tipo de dato, aumentando la solidez del programa.

b) Implementación de Estructuras de Datos Personalizadas

Para la simulación del juego, se han creado varias estructuras de datos personalizadas, cada una diseñada para cumplir con una necesidad específica del juego:

Mazo (Lista Circular Enlazada): El mazo en el juego, donde las cartas se barajan y se retiran de manera secuencial, se representa eficazmente con una lista circular enlazada. Esta estructura permite un flujo continuo de las cartas, donde la última carta se vincula a la primera, simulando la extracción desde cualquier lugar y su regreso al final. La implementación de funciones como `agregar_carta`, `sacar_carta`, `mezclar` y `repartir` garantiza una gestión dinámica y realista del mazo.

Manos de Jugadores y Historial de Jugadas (Pilas): Las manos de los jugadores, que aumentan y disminuyen al tomar y jugar cartas, se pueden ver como pilas (LIFO - Último en entrar, primero en salir). De manera similar, el historial de jugadas se conserva como una pila, lo que permite un seguimiento cronológico y la posibilidad de revertir acciones. Las funciones más importantes aquí incluyen apilar (para añadir una carta a la mano o al historial) y desapilar (para jugar una carta o revisar la última jugada).

Organización General (Listas Doblemente Enlazadas Genéricas): Para permitir una administración más adaptable de elementos dentro del juego, como la lista de participantes o algunas colecciones de cartas, se utilizan listas doblemente enlazadas genéricas. Estas estructuras permiten navegar en ambas direcciones y realizar inserciones o eliminaciones de manera eficiente en cualquier lugar, ofreciendo así una base flexible para la organización interna del sistema.

c) Programación Orientada a Objetos (POO) en la Simulación

La POO es clave para representar las entidades del juego de una manera que sea fácil de entender y ampliar. Se crean clases para ilustrar los componentes esenciales de la simulación:

Clase Carta: Representa una carta individual con características como su valor, color y tipo. Métodos relacionados pueden incluir `es_compatible_con()` o `obtener_informacion()`.

Clase Jugador: Simboliza a cada jugador en el juego, abarcando su mano (que se implementa como una Pila), su nombre y su habilidad para llevar a cabo acciones como `jugar_carta()` o `tomar_carta_del_mazo()`.

Clase Juego: Contiene la lógica principal del juego, coordinando las interacciones entre el mazo, los jugadores y las reglas. Esta clase se encarga del turno actual, la carta que está en juego y la verificación de

las condiciones de victoria. La utilización de POO permite que cada objeto actúe como una entidad independiente, interactuando a través de interfaces claramente definidas.

d) Gestión de Datos y Visualización

La conservación de los datos del juego y la representación de su estado son factores esenciales para la depuración y el análisis.

Almacenamiento de Datos (XML): Se emplea el formato XML para guardar la información de las partidas jugadas, incluyendo un resumen y una secuencia exhaustiva de cada acción (quién jugó qué carta, quién tomó una carta, etc.). Este formato organizado facilita la comprensión y el procesamiento de los datos del juego para futuras simulaciones o estudios.

Visualización (Graphviz): Para obtener una mejor comprensión de la dinámica del juego y el estado de las estructuras de datos en un momento específico, se utiliza Graphviz. Esta herramienta permite crear representaciones visuales (diagramas de grafos) del mazo, las manos de los jugadores y el desarrollo de la partida. Esto resulta especialmente útil para corregir algoritmos de barajado o para seguir el movimiento de las cartas, proporcionando una visión clara de cómo se organizan y fluyen los datos a través del sistema. La habilidad de visualizar estas estructuras complejas es invaluable para el desarrollo y la validación del sistema.

En el caso de inclusión de figuras, deben ser nítidas, legibles en blanco y negro. Se denomina figuras a gráficas, esquemas, fotografías u otros elementos gráficos.

Conclusiones

Este proyecto ha demostrado exitosamente la implementación de una simulación de juego de cartas, validando la eficacia de los Tipos de Datos Abstractos (TDA) y la Programación Orientada a Objetos (POO) como pilares fundamentales en el desarrollo de software. La creación de estructuras personalizadas como listas circulares para el mazo y pilas para las manos de los jugadores no solo optimizó el rendimiento, sino que también garantizó la abstracción y la mantenibilidad del código.

La adopción de XML para la persistencia de datos y Graphviz para la visualización del estado del juego subraya la importancia de herramientas complementarias que facilitan la depuración y el análisis. En definitiva, esta iniciativa refuerza que la comprensión y aplicación de los TDAs y la POO son esenciales para construir sistemas complejos que no solo funcionen, sino que sean escalables, comprensibles y fáciles de mantener.

Referencias bibliográficas

GeeksforGeeks. (s.f.). Circular Linked List. Obtenido de <https://www.geeksforgeeks.org/dsa/circular-linkedlist/>

GeeksforGeeks. (s.f.). Reading and Writing XML files in Python. Obtenido de <https://www.geeksforgeeks.org/python/reading-andwriting-xml-files-in-python/>

Graphviz. (s.f.). Graphviz Documentation. Obtenido de <https://graphviz.org/documentation/>

Python Software Foundation. (s.f.). The Python Standard Library: Tkinter. Obtenido de <https://docs.python.org/3/library/tk.html>

Python Software Foundation. (s.f.). The Python Tutorial. Obtenido de <https://docs.python.org/3/>

Extensión:

Figura 1: Diferentes tipos de listas usadas para la ejecución del código

```
class Carta:
    def __init__(self, color: str, numero: int):
        self.color = color
        self.numero = numero

    def __str__(self):
        return f"({self.numero}) ({self.color})"

    def __repr__(self):
        return f"Carta(color='{self.color}', numero={self.numero})"

    def __eq__(self, other): # Para comparar cartas
        if not isinstance(other, Carta):
            return NotImplemented
        return self.color == other.color and self.numero == other.numero

# Nodo para la Lista Circular Simplemente Enlazada para el mazo
class NodoCarta:
    def __init__(self, carta: Carta):
        self.carta = carta
        self.siguiente = None # Puntero al siguiente nodo

# Usada para el mazo de reserva
class ListaCartas:
    def __init__(self):
        self.primerio = None # Primer nodo de la lista
        self.ultimo = None # ultimo nodo de la lista
        self.size = 0 # Contador de elementos

    def insertar(self, carta: Carta):
        nuevo_nodo = NodoCarta(carta)
        if not self.primerio: # Si la lista esta vacia
            self.primerio = nuevo_nodo
            self.ultimo = nuevo_nodo
            self.ultimo.siguiente = self.primerio # El ultimo apunta a si mismo (circular)
        else: # Si la lista ya tiene elementos
            self.ultimo.siguiente = nuevo_nodo # El ultimo actual apunta al nuevo
            nuevo_nodo.siguiente = self.primerio # El nuevo apunta al primero (circular)
            self.ultimo = nuevo_nodo # El nuevo nodo es el ultimo
        self.size += 1

class NodoGenerico:
    def __init__(self, data):
        self.data = data
        self.siguiente = None
        self.anterior = None

class ListaGenerica:
    def __init__(self):
        self.primerio = None
        self.ultimo = None
        self.size = 0

    def agregar_final(self, data):
        nuevo_nodo = NodoGenerico(data)
        if not self.primerio:
            self.primerio = nuevo_nodo
            self.ultimo = nuevo_nodo
        else:
            self.ultimo.siguiente = nuevo_nodo
            nuevo_nodo.anterior = self.ultimo
            self.ultimo = nuevo_nodo
        self.size += 1

    def obtener_por_indice(self, index: int):
        if index < 0 or index >= self.size:
            return None

        current = self.primerio
        for i in range(index):
            if current is None:
                return None
            current = current.siguiente
        return current.data

    def get_display_string(self) -> str:
        if self.size == 0:
            return "Lista vacia."

        current = self.primerio
        items_str = ""
        count = 0
        while current and count < self.size:
            items_str += str(current.data)
            current = current.siguiente
            count += 1
            if current and count < self.size:
                items_str += " -> "
        return items_str

    def clear(self):
        self.primerio = None
        self.ultimo = None
        self.size = 0
```

```
# Pila de Cartas (NodoPila y PilaCartas)
class NodoPila:
    def __init__(self, carta: Carta):
        self.carta = carta
        self.siguiente = None

class PilaCartas:
    def __init__(self):
        self.cima = None
        self.size = 0

    def apilar(self, carta: Carta):
        nuevo_nodo = NodoPila(carta)
        nuevo_nodo.siguiente = self.cima
        self.cima = nuevo_nodo
        self.size += 1

    def desapilar(self) -> Union[Carta, None]:
        if not self.cima:
            return None
        carta_extraida = self.cima.carta
        self.cima = self.cima.siguiente
        self.size -= 1
        return carta_extraida

    def ver_cima(self) -> Union[Carta, None]:
        if not self.cima:
            return None
        return self.cima.carta

    def esta_vacia(self) -> bool:
        return self.cima is None

    def get_display_string(self) -> str:
        if self.esta_vacia():
            return "Pila vacia."

        actual = self.cima
        cards_str_parts = ""
        first = True
        while actual:
            if not first:
                cards_str_parts += " -> "
            cards_str_parts += str(actual.carta)
            first = False
            actual = actual.siguiente
        return cards_str_parts
```

Figura 2: Diferentes mezclas utilizadas para la ejecución del programa

```
def shuffle_right(self, movimientos: int):
    if self.primerio is None or self.size <= 1 or movimientos == 0:
        return

    movimientos_efectivos = movimientos % self.size
    if movimientos_efectivos == 0:
        return

    new_last_candidate = self.primerio
    for _ in range(self.size - movimientos_efectivos - 1):
        if new_last_candidate is None:
            print("DEBUG: Error en shuffle_right: Mazo inesperadamente corto al buscar new_last_candidate.")
            return
        new_last_candidate = new_last_candidate.siguiente

    new_first_candidate = new_last_candidate.siguiente if new_last_candidate else None

    if new_first_candidate is None:
        print("DEBUG: Error en shuffle_right: new_first_candidate es None.")
        return

    self.primerio = new_first_candidate
    self.ultimo = new_last_candidate
    self.ultimo.siguiente = self.primerio

    print("DEBUG: Mazo despues de RIGHT_SHUFFLE (size: (self.size)): (self.get_display_string())")

def half_shuffle(self):
    if self.size != 51:
        print("DEBUG: Half Shuffle solo aplica a mazos de 51 cartas. Tamaño actual: (self.size). No se aplico.")
        return

    print("DEBUG: Iniciando HALF_SHUFFLE (lista simplemente enlazada)...")

    seg1_fin = self.primerio
    for _ in range(24):
        if seg1_fin is None:
            print("DEBUG: Error en half_shuffle: Mazo inesperadamente corto al buscar seg1_fin.")
            return
        seg1_fin = seg1_fin.siguiente

    if seg1_fin is None:
        print("DEBUG: Error en half_shuffle: seg1_fin es None.")
        return

    pivote_node = seg1_fin.siguiente # Pivote en la posicion 25
    if pivote_node is None:
        print("DEBUG: Error en half_shuffle: pivote_node es None.")
        return

    seg2_inicio = pivote_node.siguiente # todo en la posicion 26
    if seg2_inicio is None:
        print("DEBUG: Error en half_shuffle: seg2_inicio es None.")
        return

    seg2_fin = self.ultimo

    seg1_fin.siguiente = None # quitamos los apuntadores
    pivote_node.siguiente = None

    seg2_fin.siguiente = pivote_node
    pivote_node.siguiente = self.primerio

    self.primerio = seg2_inicio
    self.ultimo = seg1_fin
    self.ultimo.siguiente = self.primerio

    print("DEBUG: Mazo despues de HALF_SHUFFLE (size: (self.size)): (self.get_display_string())")

def faro_shuffle(self):
    if self.size != 51:
        print("DEBUG: Faro Shuffle requiere 51 cartas. Tamaño actual: (self.size).")
        return

    print("DEBUG: Iniciando FARO_SHUFFLE (implementacion exacta de la logica de usuario)...")

    #Extraer todas las cartas del mazo actual a una listaGenerica
    temp_original_deck_lg = listaGenerica()
    current_node_lc = self.primerio
    for _ in range(self.size):
        if current_node_lc is None:
            print("DEBUG: Error en faro_shuffle: Mazo inesperadamente corto al copiar a temp_original_deck_lg.")
            return
        temp_original_deck_lg.agregar_final(current_node_lc.carta) # Agrega el objeto Carta
        current_node_lc = current_node_lc.siguiente

    self.clear() # Limpiar el mazo principal para reconstruirlo

    # Preparar las listas para los subgrupos usando listaGenerica
    impares_A = listaGenerica()
    pares_A = listaGenerica()
    impares_B = listaGenerica()
    pares_B = listaGenerica()
    pivote_card = Union(Carta, None) = None

    #Separar en subgrupos
    for current_idx_lg in range(temp_original_deck_lg.size): # Iterar por los indices
        posicion = current_idx_lg + 1 # Convertir a posicion 1
        carta_a_procesar = temp_original_deck_lg.obtener_por_indice(current_idx_lg)
        if carta_a_procesar is None:
            print("DEBUG: Error: Carta en indice (current_idx_lg) es None durante la separacion de Faro Shuffle.")
            return

        if posicion >= 1 and posicion <= 25: # Mitad A (original_index 0-24)
            if posicion % 2 == 1: # Posiciones impares de Mitad A
                impares_A.agregar_final(carta_a_procesar)
            else: # Posiciones pares de Mitad A
                pares_A.agregar_final(carta_a_procesar)
        elif posicion == 26: # Pivote
            pivote_card = carta_a_procesar
        elif posicion >= 27 and posicion <= 51: # Mitad B
            if posicion % 2 == 1: # Posiciones impares de Mitad B
                impares_B.agregar_final(carta_a_procesar)
            else: # Posiciones pares de Mitad B
                pares_B.agregar_final(carta_a_procesar)
```

Figura 3: Análisis del archivo XML

```
def __init__(self):
    self.__init__(self)

    def load_from_xml(self, filepath: str, log_callback=None) -> Union(Partida, None):
        print("DEBUG: Intentando cargar XML desde: (filepath)")
        try:
            tree = etree.parse(filepath)
            root = tree.getroot()

            new_partida_template = Partida()

            cartas_root_element = root.find("mazo_disponible(cartas)")
            if cartas_root_element:
                initial_deck_cards_template = listaCartas()
                loaded_card_count = 0
                MAX_DECK_SIZE = 51 # Definimos el tamaño maximo del mazo

                for carta_element in cartas_root_element.findall("carta"):
                    # Si el mazo cargado es mayor, ignorar las restantes
                    if loaded_card_count >= MAX_DECK_SIZE:
                        if log_callback:
                            log_callback("Advertencia (XML): El archivo contiene mas de (MAX_DECK_SIZE) cartas. Solo se tomaran las primeras (MAX_DECK_SIZE).")
                        print("DEBUG: Advertencia: El archivo XML contiene mas de (MAX_DECK_SIZE) cartas. Se ignoran las adicionales.")
                        break

                    color = carta_element.get("color")
                    try:
                        numero = int(carta_element.text.strip())
                        if 1 <= numero <= 13:
                            carta = Carta(color, numero)
                            initial_deck_cards_template.insertar(carta)
                            loaded_card_count += 1
                            print("DEBUG: Carta XML cargada: (carta)")
                        else:
                            if log_callback:
                                log_callback("Advertencia (XML): Carta con valor (carta_element.text.strip()) descartada (fuera de rango 1-13).")
                                print("DEBUG: Advertencia: Carta (carta_element.text.strip()) (color: (color)) fuera de rango.")
                            except (ValueError, TypeError):
                                if log_callback:
                                    log_callback("Advertencia (XML): Carta con numero invalido o faltante (carta_element.text.strip()) descartada.")
                                    print("DEBUG: Advertencia: Carta (carta_element.text.strip()) (color: (color)) con numero invalido.")

                                # Asigna la plantilla de mazo cargada a la partida:
                                new_partida_template.set_initial_deck_template(initial_deck_cards_template)
                                print("DEBUG: Mazo inicial de plantilla cargado con (new_partida_template.initial_deck_template.size) cartas (maximo (MAX_DECK_SIZE)).")
                                print("DEBUG: Total de (new_partida_template.initial_deck_template.size) cartas cargadas en la plantilla del mazo inicial (maximo (MAX_DECK_SIZE)).")
                            else:
                                if log_callback:
                                    log_callback("Advertencia (XML): No se encontro la seccion 'mazo_disponible(cartas)' en el XML.")
                                    print("DEBUG: ERROR: Seccion 'mazo_disponible(cartas)' no encontrada.")
                                return None # Devolvemos None si no hay datos de cartas.

            # Cargamos los jugadores
            jugadores_element = root.find("jugadores")
            if jugadores_element:
                players_names_list_generica = listaGenerica()
                loaded_player_count = 0
                for jugador_element in jugadores_element.findall("jugador"):
                    player_name_raw = jugador_element.text
                    player_name = str(player_name_raw.strip()) if player_name_raw is not None else ""

                    if player_name:
                        if loaded_player_count < 4: # Limitar a 4 jugadores
                            players_names_list_generica.agregar_final(player_name)
                            loaded_player_count += 1
                            print("DEBUG: Jugador (player_name) cargado desde XML.")
                        else:
                            print("DEBUG: Advertencia: Se ignora el jugador (player_name) porque ya se cargaron 4 jugadores.")
                    else:
                        if log_callback:
                            log_callback("Advertencia (XML): Se encontro una etiqueta <jugador> con nombre vacio o nulo.")
                            print("DEBUG: Advertencia: Etiqueta <jugador> con nombre vacio.")

            new_partida_template.set_player_names_template(players_names_list_generica)
            print("DEBUG: Nombres de jugadores plantilla establecidos: (players_names_list_generica.get_display_string())")
            print("DEBUG: Nombres de jugadores cargados desde XML: (players_names_list_generica.get_display_string())")
        else:
            if log_callback:
                log_callback("Advertencia (XML): No se encontro la seccion 'jugadores' en el XML.")
            print("DEBUG: Advertencia: Seccion 'jugadores' no encontrada.")

# Clase Partida y Shuffles
def __init__(self):
    self.partida_config_list_generica = listaGenerica()
    self.partida_config_list_generica.append(self)
    self.partida_element = root.find("partida")
    if self.partida_element:
        print("DEBUG: Procesando configuraciones de partida y shuffles...")
        self.partida_element.set_initial_deck_template(self)
        self.partida_name = self.partida_element.get("nombre")
        if self.partida_name:
            self.shuffles_for_current_partida_list_generica = listaGenerica()
            self.shuffles_for_current_partida_list_generica.append(self)
            self.shuffles_element = self.partida_element.find("shuffles")
            print("DEBUG: Shuffles para partida (partida_name).")
            for shuffle_element in self.shuffles_element.findall("shuffle"):
                shuffle_type = shuffle_element.get("type")
                x_value_str = shuffle_element.get("x")
                x_value = None
                if x_value_str:
                    x_value = int(x_value_str)
                    if log_callback:
                        log_callback("Advertencia (XML): Valor 'x' ((x_value)) para shuffle (shuffle_type) es invalido (debe ser >= 1). Se usara 0.")
                        print("DEBUG: Advertencia: Shuffle (shuffle_type) tiene x=(x_value), debe ser >= 1. Usando 0.")
                    x_value = 0
                except ValueError:
                    if log_callback:
                        log_callback("Advertencia (XML): Atributo 'x' invalido para shuffle (shuffle_type).")
                        print("DEBUG: Advertencia: Shuffle (shuffle_type) tiene atributo 'x' invalido.")
                shuffle_info_obj = ShuffleInfo(shuffle_type, x_value)
                self.shuffles_for_current_partida_list_generica.agregar_final(shuffle_info_obj)
                print("DEBUG: Shuffle cargado: (shuffle_info_obj)")
            else:
                if log_callback:
                    log_callback("Advertencia (XML): No se encontro la seccion 'shuffles' para la partida (partida_name).")
                    print("DEBUG: Advertencia: Seccion 'shuffles' no encontrada para partida (partida_name).")

# Clase para crear una partida
def __init__(self):
    self.partida_config_list_generica.append(self)
    self.shuffles_for_current_partida_list_generica.append(self)
    self.shuffles_for_current_partida_list_generica.append(self)
```

Figura 4: Código que maneja la generación de la vista del proyecto

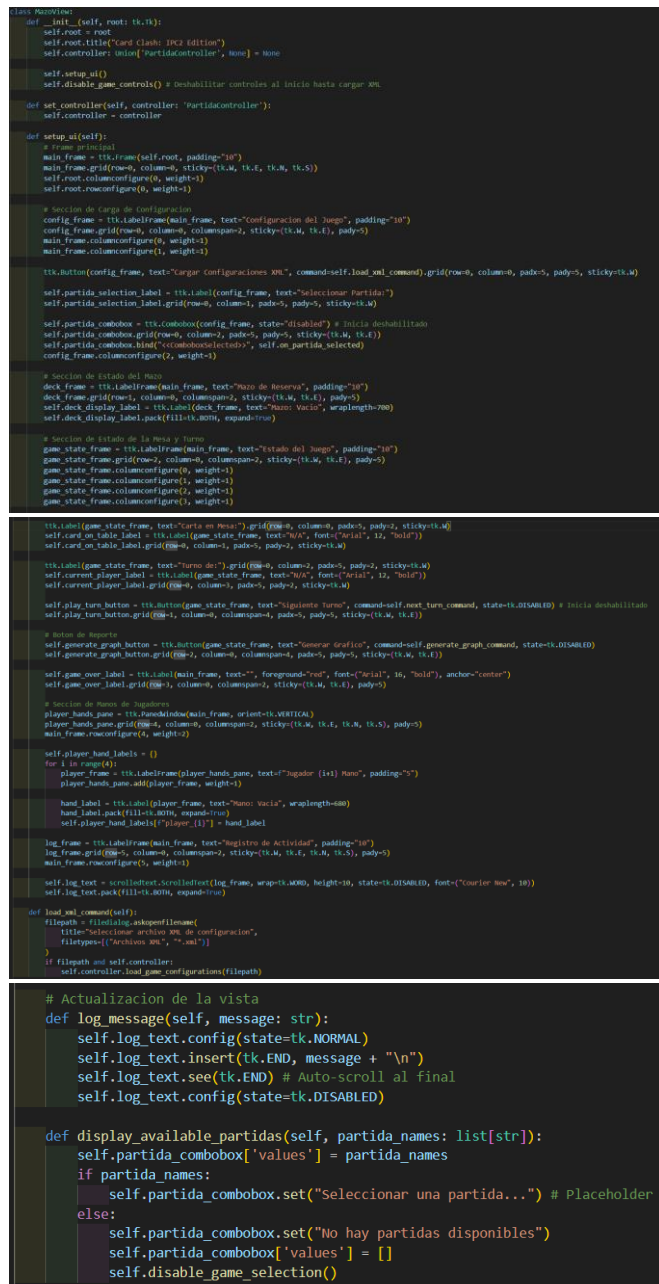


Figura 5: Generación del .dot para crear el PDF y el PNG



Figura 6: Main del proyecto

